

memdb mini-book (User Guide)

1. What we built

We built a filesystem-backed "agent memory substrate" that combines:

- Temporal graph (entities + edges with validFrom/validTo)
- Event timeline (events that justify edges)
- KV metadata (pointers, adjacency lists, manifests)
- Time-travel queries (as-of reconstruction via deltas + checkpoints + segments)
- Maintenance (health, rollups/checkpoints, segmentation)
- Pack concept: pluggable schemas and policies per agent type

This repository is a monorepo with:

- ``memdb-core`` - engine (filesystem layout + indexes + as-of + maintenance)
- ``memdb-api`` - REST API on top of core
- ``memdb-cli`` - CLI client talking to REST
- ``memdb-web`` - placeholder UI, also a REST consumer

2. The mental model

Think of memory as a *temporal knowledge graph* where every claim is a time-bounded edge and is linked to the event that introduced it.

Analogy (feature development): treat edges like "facts" in your codebase, events like "commits / reviews / chats" that justify those facts, and the as-of view like ``git checkout`` for your memory graph.

3. Packs: making memory pluggable per agent type

A **pack** is a schema + policy bundle for one agent type.

Examples:

- ``coding_assistant`` pack: entities like file/symbol/repo, predicates like depends_on/references
- ``identity_verifier`` pack: entities like subject/evidence/credential, predicates like asserts/verified_by

Packs can also define defaults like retention and "sensitive tag keys" (PII, biometrics, secrets).

4. Filesystem layout (database as folders + files)

At ``MEMDB_ROOT`` (default ``.memdb``):

- ``entities///props.json``
- ``edges//.json``
- ``events/YYYY/MM/DD/.json``
- ``kv/pointers.json`` (id -> file path)
- ``kv/adj_current/.txt`` (current adjacency)
- ``kv/adj_all/.txt`` (append-only adjacency history)
- ``deltas///...`` (delta logs for replay)
- ``kv/manifests/.json`` (checkpoint/segment metadata)
- ``segments/...`` (rollup segments)
- ``views/current/...`` and ``views/asof/...`` (materialized views)

5. Running the system (REST-first)

5.1 Start the API

From repo root:

```
deno task api
```

Defaults:

- API: `http://localhost:8787`
- set ``MEMDB_ROOT`` to change storage location

5.2 Use the CLI (REST client)

```
deno task mem init
deno task mem index
```

The CLI talks to REST at ``MEMDB_API_URL`` (default `http://localhost:8787`).

5.3 Run the placeholder web app

```
deno task -c apps/memdb-web/deno.jsonc dev
```

Open: <http://localhost:8000>

6. Key workflows

6.1 Create a pack

```
deno task mem pack create --name my_coding_pack --template coding_assistant
deno task mem packs
deno task mem pack get --name my_coding_pack
```

REST:

- `POST /v1/packs`
- `GET /v1/packs`
- `GET /v1/packs/:name`

6.2 Create entities/events/edges

```
# entity
deno task mem add-entity --pack my_coding_pack --type file --key README.md --tags lang=ts

# event (justification)
deno task mem add-event --pack my_coding_pack --kind observed --refs entity:ent_... --tag

# edge (claim)
deno task mem add-edge --pack my_coding_pack --predicate references --s ent_A --o ent_B -
```

6.3 Query neighbors (time travel)

```
deno task mem neighbors --entity ent_A --asOf 2025-12-28T12:00:00Z --pack my_coding_pack
```

REST:

- `GET /v1/neighbors?entityId=&asOf;=&pack;=`

6.4 Path query

```
deno task mem path --from ent_A --to ent_Z --asOf 2025-12-28T12:00:00Z --pack my_coding_p
```

REST:

- `GET /v1/path?from=&to;=&asOf;=&pack;=&maxDepth;=`

6.5 Maintenance and reporting

```
deno task mem health --entity ent_A --pack my_coding_pack --asOf 2025-12-28T12:00:00Z
deno task mem maintain --entity ent_A --pack my_coding_pack --asOf 2025-12-28T12:00:00Z
```

```
deno task mem maintain-all --allPacks true
deno task mem report --allPacks true --format md
```

REST:

- `GET /v1/health`
- `POST /v1/maintain`
- `POST /v1/maintain-all`
- `POST /v1/report`

7. Why we removed /rpc

`/rpc` tunneled CLI argv over HTTP. It was convenient but:

- hard to secure (argv-based authz)
- unstable contract (flags become API changes)
- bad observability (everything is "run argv")

Now:

- CLI and web share typed REST endpoints
- each endpoint can have explicit auth and metrics
- evolution is safer

8. Next directions (optional)

- Add authn/authz (API keys, per-pack permissions)
- Add bulk ingest endpoints (batch entities/edges/events)
- Add vector store integration behind an interface (embed locally, store vectors as files)
- Build the agent builder UI (create pack, seed entities, browse graphs, curate edges)

P16 Additions: Content, Search, Resolver, Vectors

Content: immutable evidence layer

Content is **source evidence**. It is never edited; it is referenced by events and facts.

REST:

- `POST /v1/content``

Body supports either:

- `{ text: "..." }` (stored as bytes, excerpt auto-generated)
- `{ base64: "..." }` (stored as bytes)
- or metadata-only content `{ uri, excerpt }`

Optional: `pack``, `mime``, `source``, `uri``, `excerpt``, `tags``.

- `GET /v1/content?pack=``
- `GET /v1/content/:id``

CLI:

```
deno task mem add-content --pack my_coding_pack --mime text/plain --source slack --text "C
```

Search: cheap but useful

Single endpoint, multiple kinds:

- `GET /v1/search?kind=entities|facts|content&q;=...&pack;=...``

CLI:

```
deno task mem search --kind facts --q timeout --pack my_coding_pack
```

This is intentionally simple (substring matching). It's a stable spine you can later replace with:

- inverted indexes,
- hybrid BM25 + vectors,
- pack-specific analyzers.

Resolver (skeleton): producing resolution artifacts

Resolution is where “world models” emerge: deciding what is canonical, superseded, corroborated, synthesized.

For now we ship a deterministic resolver that **proposes** canonical edges per edgeKey at an as-of time.

- `POST /v1/resolve``

Body: `{ entityId, asOf, pack, persist }`

If `persist=true``, the run is stored under:

- `kv/resolutions///.json``

CLI:

```
deno task mem resolve --entity ent_... --pack my_coding_pack --persist true
```

Vectors: sidecar index (bring-your-own embeddings)

We do not generate embeddings yet. Callers supply embeddings.

- `POST /v1/vectors/upsert``

`{ id, kind, pack?, embedding:number[], tags? }`

- `POST /v1/vectors/search``

`{ query:number[], topK, filter?: { pack?, kind? } }`

CLI:

```
deno task mem vectors upsert --id ent_123 --kind entity --pack my_coding_pack --embedding
deno task mem vectors search --embedding "0.1,0.2,0.3" --topK 5 --filterPack my_coding_pa
```

Where this gets you (tie-back to “two clocks”)

- **Event clock**: events + resolution artifacts + (future) decision traces as events
- **State clock**: current views + canonical facts (future) computed from resolution
- **Content**: the evidence trail that makes “why” durable

In other words: the memdb substrate can host “context graphs” with **auditable decision traces** without needing continual learning.